

CS 543 – WEB SERVICES

ASSIGNMENT 5

CampusEats HTTP Methods & Headers

Part A – Methods & the Message

A1. CampusEats Method Map

The endpoints are mapped according to the HTTP methods given in the question. GET is used for reading, POST for creating, PUT for replacing and DELETE for removing a resource.

Action	Method	URL	Purpose
List vendors	GET	/vendors	Read vendor list
View vendor	GET	/vendors/{vendorId}	Read one vendor
List menu items	GET	/vendors/{vendorId}/menu-items	Read/filter menu items
View menu item	GET	/menu-items/{itemId}	Read one menu item
Create order	POST	/orders	Create a new order
View order	GET	/orders/{orderId}	Read one order
Replace order	PUT	/orders/{orderId}	Replace complete order representation
Delete order	DELETE	/orders/{orderId}	Remove/cancel an order where permitted
Create payment	POST	/orders/{orderId}/payments	Create a payment resource
Track order	GET	/orders/{orderId}/status	Read current order status

A2. Model Non-CRUD Actions

For actions which are not normal CRUD operations, POST is used with a sub-resource. For example, cancellation can be written as /orders/{orderId}/cancel instead of putting the action in the main URL.

Action	Method	Resource-oriented URL
Checkout an order	POST	/orders/{orderId}/checkout
Cancel an order	POST	/orders/{orderId}/cancel
Set vendor availability	POST	/vendors/{vendorId}/availability

A3. Safe & Idempotent Endpoints

A safe request does not change the server state. An idempotent request gives the same final result when it is repeated.

Endpoint	Safe?	Idempotent?	Reason
GET /vendors	Yes	Yes	Read-only list operation.
GET /vendors/{vendorId}	Yes	Yes	Repeated reads do not modify the resource.

GET /vendors/{vendorId}/menu-items	Yes	Yes	Read/filter/sort/paginate only.
GET /orders/{orderId}	Yes	Yes	Reads order representation.
GET /orders/{orderId}/status	Yes	Yes	Reads current status.
POST /orders	No	No	A repeated request may create another order; use an idempotency key.
DELETE /orders/{orderId}	No	Yes	Repeated deletion leaves the resource absent.
POST /orders/{orderId}/checkout	No	No	Can trigger payment/order processing; protect with Idempotency-Key.

No GET endpoint is used to change state. The naturally retry-safe operations are the safe GET requests and resource replacement/removal operations such as PUT and DELETE. Order creation and checkout are the main operations requiring explicit duplicate-request protection.

A4. Reads Take Query Parameters

The vendor/menu collection endpoints remain pure GET reads. Filtering, sorting and pagination are represented in the query string instead of changing the URL path or using POST.

Example:

GET /vendors?campus=main&available=true&sort=name&page=1&limit=20

Example:

GET /vendors/{vendorId}/menu-items?category=beverages&available=true&sort=price_asc&page=1&limit=10

A5. OPTIONS + Allow and X-HTTP-Method-Override

For the order resource, OPTIONS can advertise the supported methods. A constrained client may use X-HTTP-Method-Override only as a documented fallback when it cannot directly send PUT or DELETE.

Example request:

OPTIONS /orders/42 HTTP

Example response:

HTTP/204 No Content

Allow: GET, PUT, DELETE, OPTIONS

Fallback example:

POST /orders/42 HTTP

Content-Type: application/json

A6. One Full HTTP Exchange

Complete request/response example for creating an order. The request explicitly states HTTP and includes the required Host, authorization, content negotiation, and JSON body headers.

REQUEST

POST /orders HTTP

Authorization: Bearer <token>

```
Content-Type: application/json
Accept: application/json
Idempotency-Key: order-2026-001
```

```
{"vendorId": "v12", "items": [{"menuItemId": "m45", "quantity": 2}], "paymentMethod": "UPI"}
```

```
RESPONSE
HTTP/201 Created
Location: /orders/987
Content-Type: application/json
Cache-Control: no-store
```

```
{"orderId": "987", "status": "PLACED", "vendorId": "v12", "items": [{"menuItemId": "m45", "quantity": 2}], "total": 120}
```

Part B – Headers on Every Message

B1. Content-Type & Negotiation

Every CampusEats request or response carrying a JSON body uses Content-Type: application/json. Clients can request JSON using Accept: application/json. If an unsupported response type is requested, the service returns 406 Not Acceptable. For large JSON responses, gzip compression may be enabled.

Header	Use
Content-Type: application/json	Identifies JSON request/response bodies.
Accept: application/json	Requests JSON representation.
Content-Encoding: gzip	Indicates compressed JSON when compression is enabled.

B2. Correct Status + Location

Situation	Status	Important header
Successful create	201 Created	Location: /orders/{orderId}
Successful read	200 OK	Content-Type: application/json
Successful delete	204 No Content	No response body
Malformed request	400 Bad Request	Content-Type: application/json
Resource missing	404 Not Found	Content-Type: application/json
State/resource conflict	409 Conflict	Content-Type: application/json
Domain/business rule rejection	422 Unprocessable Content	Content-Type: application/json

Location has two related meanings: after a 201 response it identifies the newly created resource; on a 3xx redirect it identifies the target resource/URI to which the client should go.

B3. Authorization

Protected CampusEats endpoints accept the Authorization header in Bearer form. A missing or empty token results in 401 Unauthorized. This assignment only requires header handling; a real token-generation/validation system is not required.

Example:

Authorization: Bearer <token>

B4. Cache a Read

For a single-item GET such as GET /vendors/{vendorId}, the response can include Cache-Control and an ETag. The ETag changes whenever the represented resource changes.

```
Example response:
HTTP/200 OK
Cache-Control: private, max-age=60
ETag: "vendor-v12-v7"
Content-Type: application/json
```

B5. Rate-Limit Signalling (Not yet implemented)

Rate limits are tracked per client rather than globally. The service returns the limit and remaining budget. When the client exceeds its budget, the service returns 429 Too Many Requests with Retry-After.

Header	Example
X-RateLimit-Limit	100
X-RateLimit-Remaining	12
Retry-After	30

B6. CORS

A browser application hosted on another origin requires CORS permission. The API returns Access-Control-Allow-Origin and responds to the browser's OPTIONS preflight request.

Example:

Access-Control-Allow-Origin: https://campuseats.example

Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS

Access-Control-Allow-Headers: Authorization, Content-Type, Accept, Idempotency-Key

B7. Security & General Headers

CampusEats responses include the security/general headers required by the assignment. X-Content-Type-Options prevents MIME sniffing, while Strict-Transport-Security tells compatible browsers to use HTTPS for the service in production. Date and Server may also be supplied by the HTTP framework.

Header	Purpose
X-Content-Type-Options: nosniff	Prevents MIME-type sniffing.
Strict-Transport-Security: max-age=31536000; includeSubDomains	Enforces HTTPS use in compatible clients.
Date	Identifies the response generation time.
Server	Identifies the serving HTTP software/framework when provided.